

TECHNICAL DEBT E REFACTORING

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**



Technical Debt - TD (Debito Tecnico)

- È l'insieme delle conseguenze future derivanti dalla scelta di una soluzione tecnica rapida o sub-ottimale invece di un approccio più robusto e costoso, ma più lento.
- Termine coniato nel 1992 da Ward Cunningham

In finanza:	Nello sviluppo software:
Ottieni un prestito oggi (debito)	Scrivi codice "quick & dirty"
Paghi interessi nel tempo	La manutenzione è più costosa
Puoi estinguere il debito	Puoi eseguire refactoring
Interessi composti	Complessità che si accumula
Puoi fallire se non lo restituisci	Ogni modifica ha costi e rischi sproporzionati



Classificazione del TD

- Non tutto il debito è uguale:
 - un debito prudente e consapevole può essere accettabile, se circoscritto e accompagnato da un piano di rimborso, come nel caso di un mutuo per comprare una casa.
 - un debito sconsiderato o architetturale, invece, può bloccare l'evoluzione di intere aree del sistema.

Volontarietà	Tipologia	
	<i>Prudente</i>	<i>Avventato</i>
<i>Deliberato</i>	Scelta consapevole per time-to-market	Scelta consapevole ma rischiosa
<i>Involontario</i>	Non sapevamo fare di meglio	Disattenzione o inesperienza

Le cause del TD

- Pressione temporale
- Mancanza di test → Paura di fare refactoring
- Codice non documentato → Codice incomprensibile
- Accoppiamento elevato → Modifiche a catena
- Duplicazioni di codice → N volte la stessa modifica
- Nomi poco chiari / standard di codifica non seguiti
- Funzioni troppo lunghe
- Nessuna code review
- Il progetto iniziale non è rivisto quando il sistema cresce



Sintomi di TD

- Stime poco affidabili
- Bug regressivi (molti bug sono introdotti in caso di modifiche e aggiunte)
- Task che restano in lavorazione per troppo tempo
- Build lente
- Aree del codice che nessuno vuole toccare
- Lead time crescente
- Throughput instabile
- Molto rework e molti blocchi in fase di review/test



Impatto su Kanban e Scrum

- Per un team che usa Kanban, il TD:
 - aumenta il lavoro bloccato
 - nasconde WIP effettivo
 - allunga le code
 - fa crescere il numero delle feature che violano il WIP
- Per un team Scrum, il TD:
 - erode la capacità reale
 - rende le stime meno credibili
 - la velocità può apparire costante, ma fragilità crescente
 - le riunioni di retrospettiva diventano ripetitive senza azioni tecniche strutturali



Matrice di Prioritizzazione

- Azioni in funzione dell'impatto e del costo di refactoring del TD:

Impatto del problema	Sforzo di refactoring	
	<i>Basso</i>	<i>Alto</i>
<i>Alto</i>	Esegui refactoring ORA! 	Pianifica nel Backlog 
<i>Basso</i>	Accetta e monitora 	Ignora (per adesso) 

Refactoring: lo strumento per pagare il debito

Definizione:

“Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure” (Martin Fowler)



Martin Fowler a XP2000,
Cagliari, Hotel Mediterraneo



Qualche dettaglio.....

- Il refactoring non è un'attività cosmetica: è il modo principale per evitare che gli interessi del debito distruggano il flusso di valore rilasciato al cliente
- Il refactoring è fatto per rendere il codice semplice, facile da leggere, meglio organizzato e più facile da modificare: esso migliora la struttura interna del codice.
- Non si tratta solo di ripulire il codice. Il codice viene cambiato (e ripulito) in un maniera efficiente e controllata
- Non si aggiungono funzionalità: se si aggiungono, non è refactoring!



Cosa vuole dire fare Refactoring?

- Semplificando: esso consiste in una riorganizzazione, ristrutturazione del codice (di solito ad oggetti)
 - Implica equivalenza
 - Viene modificata la struttura, non il comportamento!
 - Esempio: $(x^2 - 1) == (x+1) * (x-1) == (x * x - 1)$
 - Con l'intenzione di migliorare le cose ...
- **Non vuol dire:** “fix a bug”!
- **Non vuol dire:** stravolgere il progetto
 - Es. Cambiare l'architettura
- **Non vuol dire:** cambiare la piattaforma
 - Es. da JavaEE a .Net



Perchè “fare” Refactoring?

- Prevenire il “design decay”
 - Ripulire/migliorare il progetto
- “Clean up messes in the code”
 - Mess = confusione, disordine ...
- Semplificare il codice
- Semplificare il testing
- Aumentare la leggibilità e comprensibilità del codice
- Trovare bug
- Ripagare il TD!



Quando “fare” Refactoring?

- Non pianificate: “2 settimane di refactoring ogni 2 mesi”!
- **Fate Refactoring** quando:
 - Si vuole aggiungere una nuova funzionalità
 - il refactoring aiuta a farlo meglio e più velocemente
 - Quando si risolve un bug
 - Quando il codice “smells”(puzza)
- “Regola del tre”
 - *La prima volta che fate qualcosa, fatelo*
 - *La seconda, duplicate*
 - *La terza, eseguite il refactoring!*

Applicate il refactoring il più spesso possibile!



Regole del Refactoring

«Cercate di lasciare questo mondo un po' migliore di quanto non l'avete trovato»
(dall'ultimo messaggio di B.P. agli Esploratori)



- Regola del boy scout:
Lascia il codice meglio di come l'hai trovato
- Regola di Pareto o dell' 80/20:
80% di nuove feature, 20% di refactoring
- ***Includi la qualità del codice nella "Definition of Done"***



Vilfredo Pareto
(1848-1923)



Code Smell

- **Indicatore** che qualcosa nel codice non va
 - Indicatore, non certezza!
 - Potrebbe essere solo un problema di stile, oppure qualcosa che riduce la comprensione, oppure nascosto c'è un problema più grave ...
- Esistono strumenti che calcolano le metriche del software e che ci indicano quali sono i **code smell** presenti
 - *Porzioni di codice da migliorare*
 - Ad esempio, *Pylint* che ci avverte quando un metodo è troppo lungo.



Esempi di code smell

- Troppo codice:
 - Large method
 - Large class
 - **Duplicated code (cloni)**
 - Code that is not executed
 - Long parameter List
 - Refused Bequest: sottoclasse che non richiede dati o metodi della superclasse
 - Feature Envy: metodo che usa troppi metodi di altre classi
 - Temporary Field
 - Strikingly Similar Subclasses
 - Expensive Set Up
 - Unused Code



Esempi di code smell

- Non abbastanza codice:
 - Classes with little code
 - Lazy Class
 - Empty catch clauses (senza codice “except”)
 - Incomplete Library Class
 - Data class
 - Contiene solo variabili d'istanza
- Al di fuori del codice
 - Poor Naming
 - Magic Numbers
 - Primitive Obsession
 - Excessive commenting
 - Excessive Logging



Esempio: funzione per calcolo prezzo finale con sconti per età e coupon

```
def proc(u, a, t, c):
    b = 0
    # 1. Magic Numbers &
    # 2. Primitive Obsession

    if t == 1:
        b = 100
    elif t == 2:
        b = 200
    elif t == 3:
        b = 300

    # 3. Nested Conditionals

    if a < 18:
        b = b * 0.5
    else:
        if c == 'SI':
            b = b - (b * 0.1)
```

```
# 4. Duplicate Code (logica
#     coupon ripetuta)
if a > 65:
    if c == 'SI':
        b = b - (b * 0.1)
    else:
        b = b * 0.8
```

```
# 5. Long Method (fa troppo)
if b < 0:
    b = 0
print("Prezzo: " + str(b))
return b
```

```
# 6. Poor Naming (u, a, t, c, b)
```



Code Smells Identificati

- 1) Poor Naming:** Variabili criptiche (u, a, b).
- 2) Magic Numbers:** Valori “scalpellati” (100, 0.5, 18, 65,...).
- 3) Nested Conditionals:** Troppi if annidati.
- 4) Duplicate Code:** Controllo coupon `c == 'SI'` ripetuto.
- 5) Long Method:** La funzione calcola, valida e stampa.
- 6) Primitive Obsession:** Usa interi per tipi prodotto (`t == 1`).



Applicazione delle Tecniche di Refactoring

- 1) Rename Variable/Method:** Nomi descrittivi.
- 2) Replace Magic Numbers with Constants:** Costanti con nome.
- 3) Decompose Conditional:** Estrazione logica sconti.
- 4) Remove Duplicated Code:** Logica coupon centralizzata.
- 5) Introduce Parameter Object / Enum:** Tipi prodotto strutturati.
- 6) Separate Query from Modifier:** Rimosso *print* dalla logica.



Il codice dopo il refactoring

```
PREZZI_BASE = {STANDARD: 100, PREMIUM: 200, VIP: 300}  
SCONTO_MINORENNE = 0.5  
SCONTO_ANZIANI = 0.8  
SCONTO_COUPON = 0.1
```

```
def calcola_prezzofinale(tipo_prodotto, eta, ha_coupon):  
    prezzo = PREZZI_BASE.get(tipo_prodotto, 0)  
    prezzo = applica_sconto_eta(prezzo, eta)  
    prezzo = applica_sconto_coupon(prezzo, ha_coupon)  
    return max(0, prezzo)
```

```
def applica_sconto_eta(prezzo, eta):  
    if eta < 18:  
        return prezzo * SCONTO_MINORENNE  
    if eta > 65:  
        return prezzo * SCONTO_ANZIANI  
    return prezzo
```

```
def applica_sconto_coupon(prezzo, ha_coupon):  
    if ha_coupon:  
        return prezzo * (1 - SCONTO_COUPON)  
    return prezzo
```

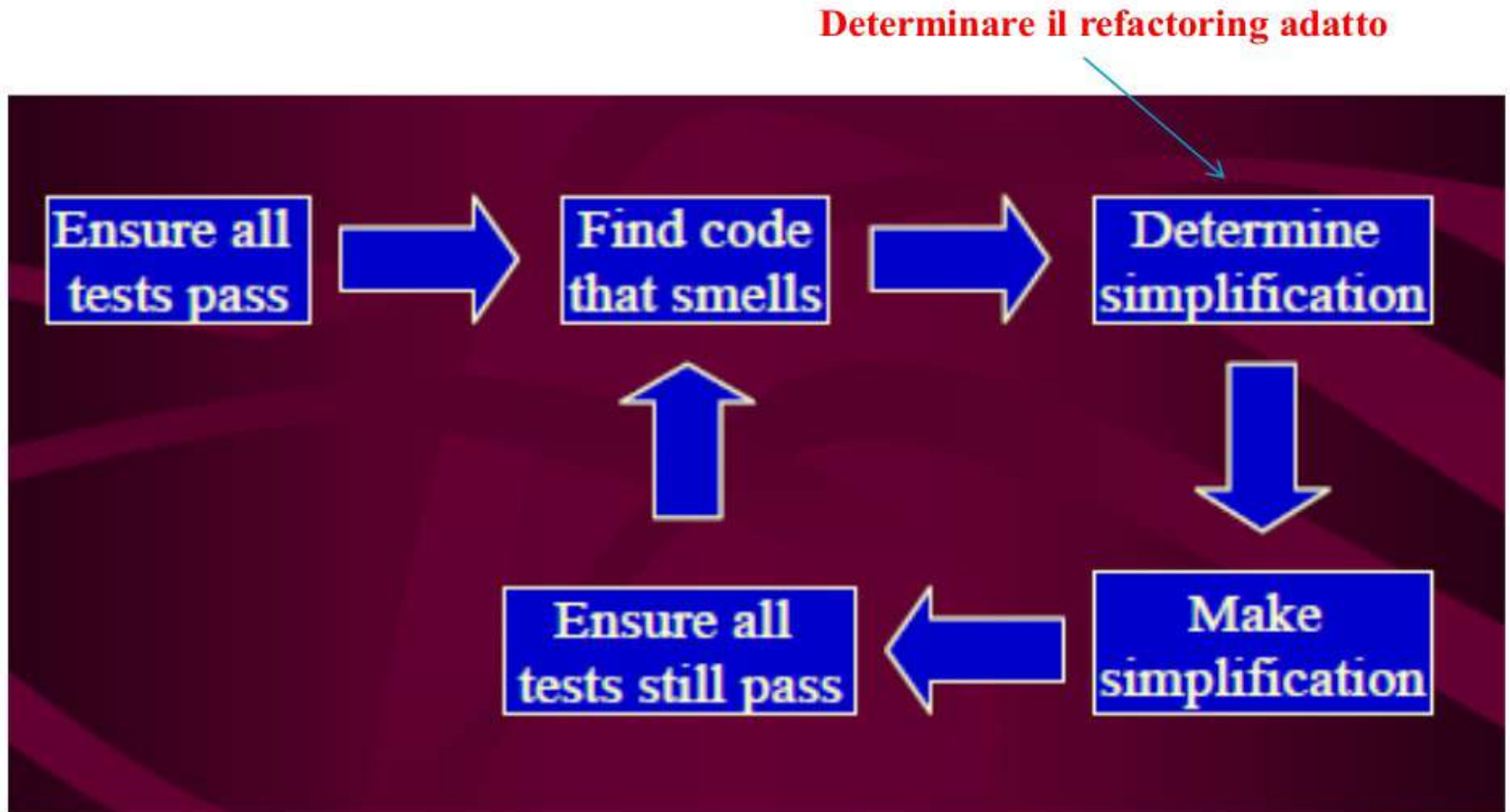


Risultato

- Leggibilità migliorata del 100%
- Testabilità aumentata (funzioni piccole)
- Manutenzione semplificata (cambiamenti localizzati)
- Il Refactoring non cambia il comportamento, ma migliora la struttura interna
- Regola d'Oro:
 - *"Il codice è letto molto più spesso di quanto viene scritto."*



Processo di refactoring



La lezione più importante

Il “ritmo” del refactoring:

- Modifica (piccola) del codice
 - Quando i test “passano”
- Seguendo una procedura definita in modo preciso
 - Catalogo dei refactorings
- Compilare ed eseguire i test
- Morale:
 - Non fare il passo più lungo della gamba...



Catalogo dei Refactoring

- Molti dei refactoring sono “molto piccoli/semplici”
 - low-level refactoring
 - Rename method
 - Extract method
- I low-level refactoring sono i building-blocks per (o che abilitano) i refactoring più complessi ...
 - Es. quelli che introducono i Design pattern

Name	Description
Expand Accessors	Expands single-line getter or setter code onto multiple lines.
Expand Assignment	Expands this short form assignment to a full assignment.
Expand Getter	Expands single-line getter code onto multiple lines.
Expand Lambda Expression	Converts a lambda expression to an equivalent anonymous function.
Expand Null Coalescing O...	Converts a null coalescing operation to an equivalent ternary expression.
Expand Setter	Expands single-line setter code onto multiple lines.
Expand Ternary Expression	Expands active ternary expression to the if statement.
Extract ContentPlaceholder	Moves the selected content from a .master page to a new page.
Extract ContentPlaceHold...	Moves the content that is *outside* of the selection (in the file) to the new page.
Extract Function	Creates a new function within the enclosing namespace (or class).
Extract Interface	Generates a new interface from the public members of this class.
Extract Method	Creates a new method from the selected code block. The selected code is moved to the new method.
Extract Method to Type	Creates a new method from the selected code block and moves it to the specified type.
Extract Property	Creates a new property from the selected code block. The selected code is moved to the new property.
Extract Script	Extracts JavaScript code to an external file.
Extract String to Resource	Extracts this string to a resource file.
Extract String to Resourc...	Extracts all matching strings in the file to a resource file.
Extract Style (class)	Converts an inline style to a named class style.
Extract Style (id)	Converts an inline style to a named ID style.
Extract to XAML Resource	Extracts this string to a XAML resource file.
Extract to XAML Resourc...	Extracts all matching strings in the file to a XAML resource file.
Extract to XAML Template	Moves this template to the resource section of the file.
Extract UserControl	Creates a UserControl for the selected block including content.
Extract XML Literal to Res...	Extracts this XML literal to a resource file.
Flatten Conditional	Makes simplification of the selected condition statement.
For to ForEach	Converts a for loop into a foreach loop.
ForEach to For	Converts a foreach loop into a for loop.
Initialize Conditionally	Moves the variable initialization to an else block of the substatement.
Inline Alias	Replaces all references to a type or a namespace alias with the full name.

Lista molto lunga, in continua espansione ...

Due possibilità:

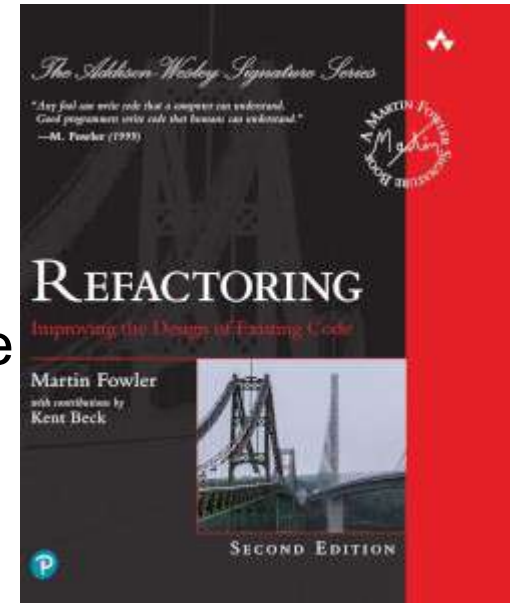
- Eseguire **manualmente** i refactoring
- Utilizzare dei tool di supporto
- Chiaramente usare dei tool è la soluzione più comoda
 - Se cambio il nome ad una classe quanti cambi devo fare nel codice?
 - Almeno il costruttore
 - Il nome del file
 - Tutte le altre classi che la usano
 - ...

*In entrambi i casi **bisogna**
controllare ri-eseguendo i casi di test!*



Tipi di refactoring

- **Controllare il libro di Fowler:**
- **Composing Methods:** la forma più semplice di Refactoring è cambiare la struttura di un metodo. Questo tipo di refactoring coinvolge la composizione di due o più metodi in uno, o l'estrazione di uno o più meto da un metodo troppo complesso.
- **Moving Features Between Objects:** spostare un metodo o una variabile d'istanza tra diverse classi, estrarre una classe da una esistente, mettere in ordine una classe.
- **Organizing Data:** creare nuovi oggetti anziché campi sparpagliati o altre strutture.
- **Simplifying Conditional Expressions**



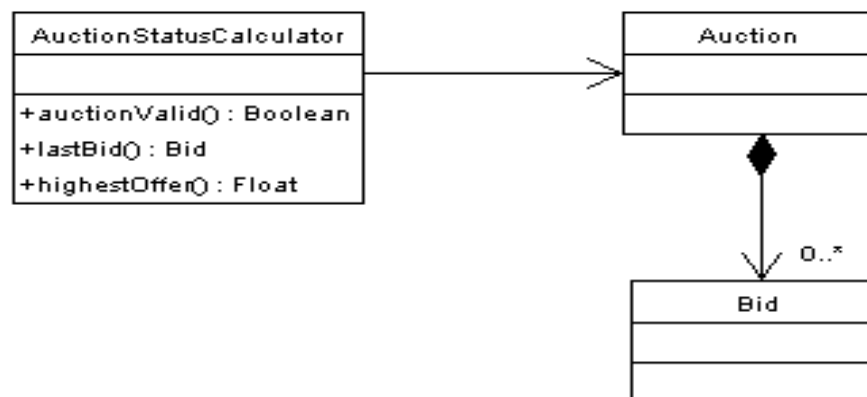
Tipi di refactoring

- **Making Methods call simpler:** rinominare un metodo, aggiungere o rimuovere un metodo; aggiungere o rimuovere un parametro a/da un metodo; rimpiazzare un parametro con una chiamata a un metodo; rimpiazzare un errore nel codice con un esplicito “lancio” dell'eccezione.
- **Dealing with Inheritance:** tirando verso l'alto o spingendo verso il basso della gerarchia una variabile di istanza o un metodo; estrarre una superclasse, una sottoclasse, o un'interfaccia; sostituzione dell'ereditarietà con delega, o viceversa.



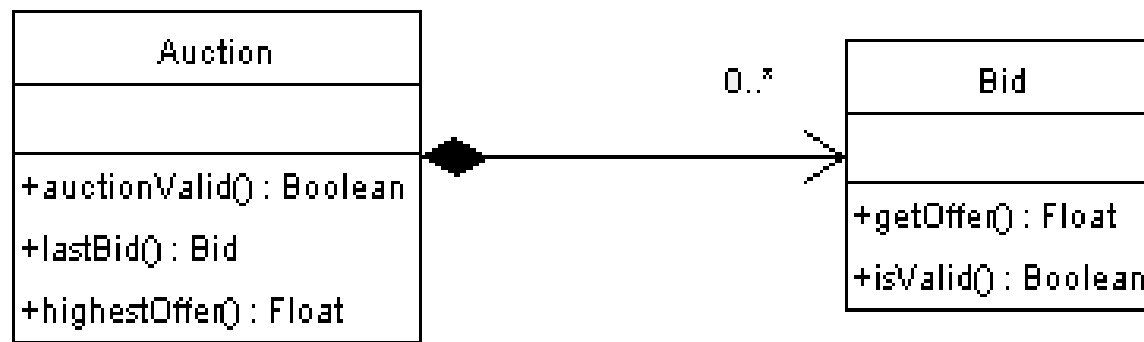
Convert Procedural Design to Objects

- Questo tipo di refactoring ha a che fare con "oggetto procedurale", che cerca di fare tutto, usando altri oggetti come una struttura di dati semplice.
- Le classi il cui nome contiene "*manager*" o "*calcolatore*" sono candidati naturali da sottoporre a controllo per questo tipo di refactoring.
- Un esempio: le classi "*Auction*" e "*Bid*", agiscono solo come strutture ("struct") che tengono i loro dati. Una classe "*AuctionStatusCalculator*" legge questi dati, calcola e restituisce le informazioni richieste.



Convert Procedural Design to Objects

- L'alternativa object-oriented, per fare gli stessi calcoli:
- Qui ci liberiamo dell'oggetto procedurale, e mettiamo la parte di computazione vicino ai dati su cui operano, cioè nelle classi “*Auction*” and “*Bid*”.



Extract Hierarchy

- Questo refactoring dovrebbe essere usato quando si dispone di una classe i cui metodi fanno ampio uso delle istruzioni condizionali "switch".
- L'uso di dichiarazioni "switch" non è OO, ma deve essere sostituito con il polimorfismo.



Tools

- Alcuni IDE hanno incorporato alcuni refactoring:
 - Es. Eclipse, PyCharm, VisualStudioCode
- Esistono poi tool o plug-in specifici che suggeriscono refactoring più complessi
 - JDeodorant (plug-in Eclipse) per Java che trova alcuni Smell e applica i refactoring associati (a richiesta dell'utente)
 - Flake8: Rileva errori logici e violazioni di stile Python (PEP8)
 - Zencoder: AI per ottimizzare il codice Python e generare refactoring

